

Assignment 4

The CEO has acquired a company working on something similar in hopes that it will make your project sell better. You meet the other team in an awkward meeting, well let's figure this out...

- Your team and the other team in the room are now one team. Assignments are done by, and grades will be awarded to the new team.
- First order of business is to get familiar with each other. Pick an event that you can do together and do it. This may be as simple as a walk in the park, but going out for a cup of something, a party, bowling, DnD at the guild hall are all popular choices. Yes, we did just include a party assignment, have fun. Yes, part of the grade. Pics or it didn't happen.
- The two leads will split into two roles: that of team lead and that of technical lead. Technically one lead is enough to demo the project, but they must be able to respond to any question and fix any issue. These responsibilities are proposed to help with the transition; you may change them if you wish.
- Team lead responsibilities:
 - Communication and task allocation on **what** needs to be done
 - They tend to know everything about all functionalities
 - Presenting the work during the demo
 - Sending the lab assistant the breakdown of the grade, the actual breakdown can be done democratically or autocratically based on team needs. (around 12-14)
 - Report creation to demonstrate that everyone worked non negligible amounts.
 - Doing regular tasks
- Technical lead responsibilities:
 - Communication and task allocation on **how** things are done
 - They tend to know all technical details about the project
 - Code reviews (pull requests)
 - Defines the code conventions/style for the team
 - Code quality assurance:
 - readability
 - unit testing
 - adherence to conventions / coding style
 - Deployments to your server
 - Doing regular tasks
- Create a new GitHub repository with the name: UBB-SE-2026-[your subgroup] (e.g. UBB-SE-2026-921-1). Invite the clients (imre.zsigmond@ubbcluj.ro and your lab assistant) as a collaborator. When adding to this project make sure to only add source files and not files generated by your IDE.
- Technical lead shall create a coding style/convention/standard document that defines how everybody should write their code in the team. There are numerous versions online that you can adopt, alter or create your own from scratch. It must contain at least 15 rules but not more than 30. A rule might be: "Use PascalCase for class names and method names." If your rules conflict with stylecop then change stylecop. Create a code review checklist document to help with staying consistent.

- Merge your solution with the other team in the room. Copy your code to the new GIT repo and merge the two structures into one. The new structure must be uniform for all classes. Eg. if a project used the model as if it was a service and the other had a separate service then make sure you have a separate service. Make sure that there is a unified GUI that can access all functionalities and not have two separate apps. Merely opening up a new window counts as separate apps, so make sure everything can be done in one window with occasional popups if needed. Make any changes to the GUI that facilitates this. If your team's code used mock functions to get data from the other team's application, then use the real functions now. Use this opportunity to purge unused code / duplicated tables. Functionalities for the other half of the project should still use mocks.
- If there are unfinished/buggy functionalities that prevent or make the merge harder, you should finish them. You may do this for any of the original projects, so if you decide that one of the original projects needs more attention you should allocate your efforts accordingly.
- Switch your data access layer (repository) to use entity framework core. Use "code first" and migrations to create your database. The entire project will use the same database and not two or more different ones. You may still save/load files for anything that is not domain related (e.g. sound/video recordings) but raw SQL / stored procedures / DB views are gone. Links between domain objects must be modelled through classes, so if class A has a class B then there would be a property of type B in class A and not an "IdOfB".
- Put your new repository on your server, with the help of an ASP.NET core web API project that runs separately from the main application. You will need "ASP.NET and web development" in the installer or go Tools -> Get Tools and Features. This does mean that your application will not connect to the database directly, but through this web API. All non-domain related files must be stored on this server as well. If you did not have your 4th seminar before you demo your 4th assignment, then present it with your web API project running locally and deploy it during your seminar.

Ongoing requirements:

- The team lead will present a report before demoing their work on the contributions of each team member, ideally task/git/chat/sent files history etc. With the scope of proving that each team member did some non-negligible amount of work.
- Your solution must not contain any business logic in its UI or its data access layer. As a reminder, MVVM/API is just the UI layer.
- Code readability, layering, style cop, test quality and coverage rules remain.
- Style cop rules may be changed by technical lead to reflect coding convention.

Optional:

- Try to start early and target getting everything done a couple of days before the deadline.
- Use tasks for work tracking and management.

Deadline: lab 5